

CDSS

Clinical Decision Support System

Full Project Architecture & Technical Documentation

Frontend · Backend · AI Engine · Database · Workflow

Version:

1.0.0

Date:

04 August 2026

Stack:

React · FastAPI · SQLite · OpenRouter AI

CDSS

Clinical Decision Support System

Full Project Architecture & Technical Documentation

Frontend · Backend · AI Engine · Database · Workflow

Version:

1.0.0

Date:

04 August 2026

Stack:

React · FastAPI · SQLite · OpenRouter AI

Table of Contents

- 1 Project Overview
 - 1.1 Purpose & Goals
 - 1.2 Key Features
 - 1.3 Technology Stack Summary
- 2 System Architecture
 - 2.1 High-Level Architecture Diagram
 - 2.2 Component Interaction Flow
- 3 Frontend
 - 3.1 Tech Stack & Configuration
 - 3.2 Page Breakdown
 - 3.3 Design System & Styling
 - 3.4 State Management & Data Flow
- 4 Backend
 - 4.1 FastAPI Server
 - 4.2 API Endpoints
 - 4.3 Request / Response Schemas
 - 4.4 Middleware & CORS
- 5 AI & Machine Learning Engine
 - 5.1 AI Pipeline Overview
 - 5.2 OpenRouter API (Primary)
 - 5.3 Local Ollama / Gemma2 (Fallback)
 - 5.4 Rule-Based Engine (Offline Fallback)
 - 5.5 NLP Extraction Logic
 - 5.6 Prompt Engineering
 - 5.7 Disease Knowledge Base
- 6 Database
 - 6.1 Database Engine & ORM
 - 6.2 Schema & Models
 - 6.3 CRUD Operations
 - 6.4 Client-Side Fallback Storage
- 7 Red Flag Detection Engine
- 8 NHM Treatment Protocol (RAG)
- 9 End-to-End Workflow
 - 9.1 Consultation Workflow
 - 9.2 Data Save Workflow
 - 9.3 Patient History Workflow
- 10 Environment & Configuration
- 11 Security Considerations

12 Known Limitations & Roadmap

1. Project Overview

The **Clinical Decision Support System (CDSS)** is a full-stack, AI-powered web application designed to assist clinicians and healthcare practitioners in making faster, more accurate diagnostic decisions at the point of care. Built for Indian primary health settings, it integrates with the National Health Mission (NHM) treatment protocols and provides real-time differential diagnoses, emergency red-flag alerts, and evidence-based treatment recommendations.

1.1 Purpose & Goals

- **Goal 1 – AI Differential Diagnosis:** Given patient symptoms, vitals, and free-text clinical notes, produce a ranked list of up to 5 probable diagnoses with confidence scores.
- **Goal 2 – NLP Extraction:** Parse unstructured clinical free-text into structured data fields (age, sex, symptoms, vitals, duration).
- **Goal 3 – Red-Flag Detection:** Instantly alert clinicians to life-threatening conditions such as meningitis, ACS, shock, and hypoxia.
- **Goal 4 – NHM Treatment Guidelines:** Retrieve and display evidence-based drug dosage and referral instructions aligned with the National Health Mission.
- **Goal 5 – Patient Record Management:** Save, retrieve, and filter patient diagnostic history in an SQLite database.

1.2 Key Features

Feature	Description
AI Cascade Inference	Primary → OpenRouter (Gemma 4 26B) → Local Ollama (Gemma2 2B) → Rule Engine
Free-Text NLP	Clinical notes parsed to extract symptoms, vitals, demographics via LLM + regex
Differential Diagnosis	Up to 5 ranked diagnoses with confidence % and SHAP-style explanatory features
Red Flag Alerts	Rule-based vital sign & symptom checks triggering emergency warnings
NHM Treatment RAG	Knowledge-base lookup returning NHM-aligned drug, dosage & referral data
Patient History	SQLite persistence with localStorage fallback; search, filter by status
Responsive UI	Liquid-glass design system; works on desktop & mobile; dark mode only
Offline Fallback	Full rule-based pipeline works without any external AI API

1.3 Technology Stack Summary

Layer	Technology	Version / Notes
-------	------------	-----------------

Frontend Framework	React + TypeScript	v19.x (Vite build)
Frontend Routing	React Router DOM	v7.x
Frontend Charts	Recharts	v3.x
Frontend Icons	Lucide React	v1.x
Frontend Animations	Framer Motion	v12.x
Backend Framework	FastAPI	≥0.111 (Python 3.x)
ASGI Server	Uvicorn	≥0.29 with standard extras
Data Validation	Pydantic	v2.x
ORM	SQLAlchemy	v2.x
Database (default)	SQLite	File: cdss.db
Database (optional)	PostgreSQL	Via psycopg2-binary
AI Primary	OpenRouter API	Model: google/gemma-4-26b-a4b-it:free
AI Fallback 1	Ollama (local)	Model: gemma2:2b @ localhost:11434
AI Fallback 2	Rule-based engine	Pure Python, no external dependencies
Build Tool	Vite	v8.x
HTTP Client (BE)	requests	≥2.31
Environment	python-dotenv	≥1.0

2. System Architecture

2.1 High-Level Architecture Diagram

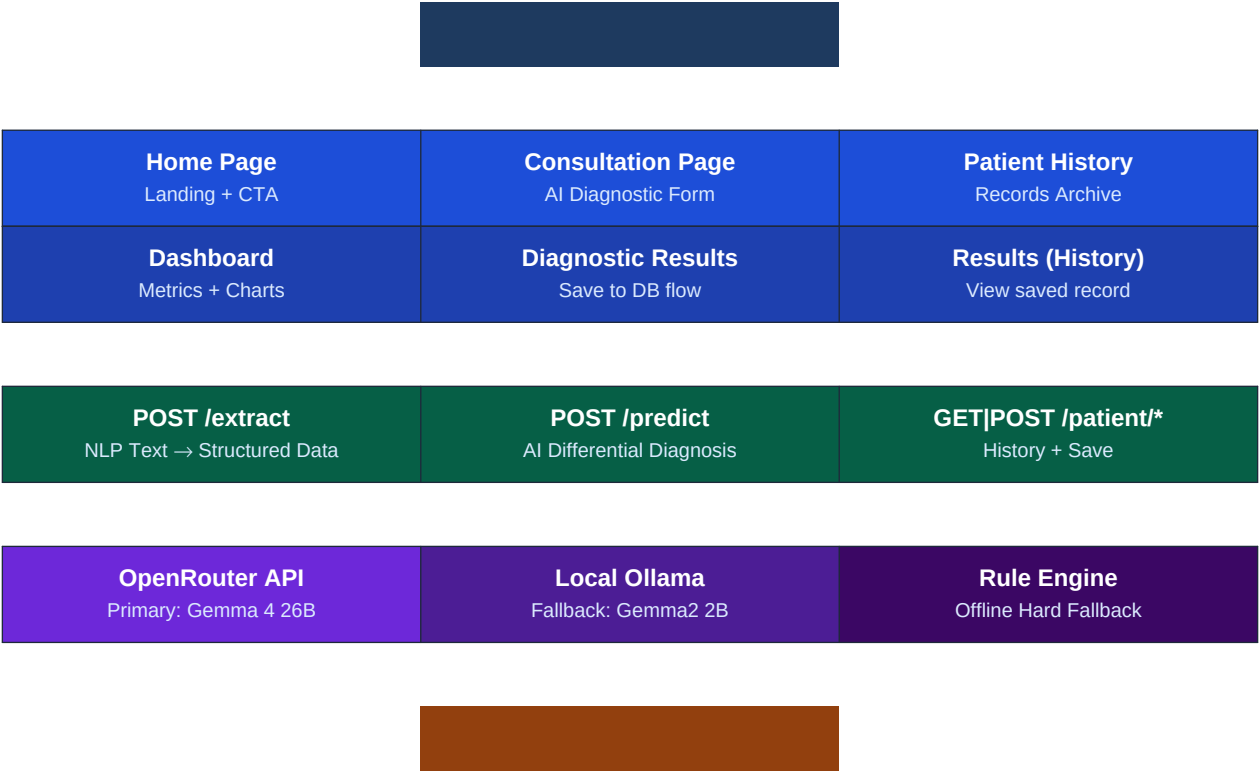


Figure 1 – CDSS High-Level Architecture: User → React Frontend → FastAPI Backend → AI Cascade (OpenRouter → Ollama → Rule Engine) → SQLite Database

2.2 Component Interaction Flow

Step	Actor	Action	Protocol
1	Clinician	Opens browser, navigates to /consultation	—
2	Frontend	Renders Consultation form (React)	—
3	Clinician	Enters patient details, symptoms, vitals, notes	UI Interaction
4	Frontend → Backend	POST /extract with free-text clinical note	HTTP/JSON
5	Backend	Calls NLP extraction (LLM or regex fallback)	Internal
6	Backend → Frontend	Returns ExtractedData (symptoms, vitals, age...)	HTTP/JSON
7	Frontend	Auto-fills form fields from extracted data	React State

8	Frontend → Backend	POST /predict with full patient payload	HTTP/JSON
9	Backend → AI	Calls OpenRouter API with structured prompt	HTTPS/JSON
10	AI → Backend	Returns JSON array of differential diagnoses	HTTPS/JSON
11	Backend	Detects red flags, retrieves NHM treatments	Internal
12	Backend → Frontend	Returns complete DiagnosticResult object	HTTP/JSON
13	Frontend	Stores result in sessionStorage, navigates to /diagnostic-results	
14	Clinician	Reviews results, clicks 'Save Diagnostic to History'	UI Click
15	Frontend → Backend	POST /patient/save to persist in SQLite	HTTP/JSON
16	Frontend	Also mirrors record to localStorage	Browser API

3. Frontend

The frontend is a **Single Page Application (SPA)** built with **React 19 + TypeScript** and bundled with **Vite 8**. It runs on **http://localhost:5173** during development. All pages are rendered client-side; routing is handled by React Router DOM v7 using nested routes under a shared Layout component.

3.1 Tech Stack & Configuration

Item	Value
Entry Point	frontend/index.html → src/main.tsx → src/App.tsx
Dev Server Port	5173 (Vite default)
Backend URL Env	VITE_BACKEND_URL=http://localhost:8000 (frontend/.env)
Font Imports	Plus Jakarta Sans + Inter (Google Fonts CDN)
CSS Strategy	Vanilla CSS with design tokens (CSS custom properties in index.css)
Type Checking	TypeScript ~6.0 with strict tsconfig
Linting	ESLint 10 with react-hooks + react-refresh plugins

3.2 Page Breakdown

■ Layout.tsx (components/)

Persistent shell around every page. Renders: (a) full-screen video background on Home, dark mesh gradient on inner pages, (b) liquid-glass navigation pill with links to Home / AI Diagnostic / Patient History, (c) account icon linking to Dashboard, (d) hamburger mobile menu with backdrop-blur overlay, (e) for child routes. Uses `useLocation()` to detect the home page and conditionally render the background video vs. the ambient gradient.

■ Home.tsx (pages/)

Landing page with hero headline 'Refining Wellness Through Natural & Precision Care', a CTA button linking to `/consultation`, and a bottom statistics bar showing a practitioner count and a live engine status pulse indicator. Pure presentational—no backend calls.

■ Consultation.tsx (pages/)

The core AI diagnostic form (652 lines). Sections: (1) Free-Text Clinical Note textarea with an 'Extract with AI' button that calls `POST /extract`; (2) Patient Demographics (Name, Age, Sex); (3) Symptom Checkboxes (18 pre-defined symptoms from NHM list); (4) Vitals panel (Temperature, BP, Pulse, SpO2, RR); (5) Lab Values (Hb, WBC, Platelets, RBS); (6) Additional Notes. On submit, calls `POST /predict`. If backend is unreachable, a

client-side `buildMockResult()` function provides a local offline fallback. Result is written to `sessionStorage` and user is navigated to `/diagnostic-results`.

■ `DiagnosticResults.tsx` (pages/)

Displays the full AI diagnostic result from `sessionStorage`. Sections: Red Flag Alerts (red glass cards), Primary Diagnosis Spotlight with confidence bar, Differential Diagnosis Chart (Recharts horizontal BarChart), Differential Diagnoses list with NHM codes, NHM Treatment Guidelines list. Has a 'Save Diagnostic to History' button that calls `POST /patient/save` AND also writes to `localStorage` for instant UI update. Navigates to `/history` after saving.

■ `Results.tsx` (pages/)

A read-only version of `DiagnosticResults`, used when a clinician clicks a row in Patient History. Loads the same `sessionStorage` payload but does NOT have the save-to-database button. The 'Back' button returns to `/history`.

■ `PatientHistory.tsx` (pages/)

Patient records list fetched from `GET /patient/history`, merged with any additional records in `localStorage` (saved_patients key). Features: search by name or ID, filter by status (all / critical / moderate / stable), stat cards for total / critical / moderate counts, a tabular list with patient identity, last visit date, visit count, last diagnosis, and colour-coded status badge. Clicking a row stores the patient data in `sessionStorage` and navigates to `/results`.

■ `Dashboard.tsx` (pages/)

Analytics dashboard at `/dashboard` (linked from the account icon). Shows 4 metric cards (Today's Consultations, Red Flag Alerts, Avg. AI Accuracy 91.3%, Patients Registered) sourced from `GET /patient/history` + `localStorage`. A Recharts AreaChart displays static weekly consultation / alert data. Two CTA buttons: Start AI Diagnostic and View Patient History.

3.3 Design System & Styling

The visual language is defined entirely in `frontend/src/index.css`. There is no Tailwind or third-party CSS framework—utility classes used in JSX (e.g. *liquid-glass*, *liquid-input*) are hand-crafted in this stylesheet.

Token / Class	Value / Purpose
<code>--bg-primary</code>	<code>#050d1a</code> — Deep navy page background
<code>--accent-blue</code>	<code>#3b82f6</code> — Primary interactive blue
<code>--accent-emerald</code>	<code>#10b981</code> — Success / stable status
<code>--accent-amber</code>	<code>#f59e0b</code> — Warning / moderate status

--accent-red	#ef4444 — Error / critical / red flag
.liquid-glass	backdrop-filter: blur(12px) + rgba(255,255,255,0.06) bg + white border — glassmorphism effect on all
.liquid-input	Glass-styled text input — same blur + border treatment as cards
.bg-mesh	CSS radial-gradient ambient mesh used as background on inner pages
Font stack	Plus Jakarta Sans → Inter → system-ui (both imported from Google Fonts)
Animation	CSS keyframe animations for pulse dots, spin loaders, and hover transitions

3.4 State Management & Data Flow

- **Local component state (useState):** form fields, loading flags, extraction results, filter values.
- **sessionStorage:** The key `cdss_result` is used to pass the full diagnostic result from Consultation → DiagnosticResults → Results pages without prop drilling or a global store.
- **localStorage:** The key `saved_patients` acts as a client-side cache that ensures Patient History and Dashboard show records even when the backend is temporarily unreachable.
- **No Redux / Zustand:** The app intentionally avoids a global state manager; navigation + storage APIs are sufficient given the current page count.
- **Data fetching:** All API calls use the native `fetch()` API with `async/await`. The BACKEND base URL is sourced from `import.meta.env.VITE_BACKEND_URL`.

4. Backend

The backend is a single-file **FastAPI** application (**backend/main.py**, 911 lines). It is served by **Uvicorn** on **http://0.0.0.0:8000** with hot-reload enabled in development. All heavy AI logic, NLP extraction, red-flag detection, and database persistence are handled here.

4.1 FastAPI Server

Attribute	Value
Entry point	backend/main.py
App title	CDSS API
Version	1.0.0
Run command	uvicorn main:app --host 0.0.0.0 --port 8000 --reload
CORS policy	allow_origins=["*"], allow_methods=["*"], allow_headers=["*"]
Dependency injection	SQLAlchemy Session via FastAPI Depends(get_db)
Startup action	Base.metadata.create_all(bind=engine) — auto-creates tables
Environment vars	Loaded from backend/.env via python-dotenv at startup

4.2 API Endpoints

Method	Path	Purpose	Auth
GET	/	Health check — returns version	None
POST	/extract	NLP extraction from free text	None
POST	/predict	Differential diagnosis + red flags + treatments	None
POST	/patient/save	Insert or update patient record	None
GET	/patient/history	Retrieve all patient records (newest first)	None
DELETE	/patient/history	Clear all patient records	None

4.3 Request / Response Schemas (Pydantic)

Schema	Direction	Key Fields
ExtractionRequest	Request	text: str

ExtractedData	Response	age, sex, symptoms[], vitals{}, duration, chiefComplaint, confidence, rawEntities[]
PredictRequest	Request	name, freeText, age, sex, temperature, bp, pulse, spo2, rr, symptoms[], hb, wbc, plat
PredictResponse	Response	patient{id,name,age,sex}, symptoms[], vitals{}, labs{}, redFlags[], diagnoses[], treatm
SavePatientRequest	Request	id, name, age, sex, condition, status
SavePatientResponse	Response	status, message, patient{id,name,age,sex,condition,status,lastVisit}

4.4 Middleware & CORS

CORSMiddleware is configured with `allow_origins=[*]` for development. This permits the React dev server (port 5173) to communicate with the FastAPI server (port 8000) without browser CORS errors. In production, this should be tightened to the specific frontend domain. No authentication middleware is currently configured — this is a known limitation (see Section 11).

5. AI & Machine Learning Engine

The AI engine is a three-tier cascade. The system always attempts the highest-quality model first and gracefully degrades to ensure a response is always returned, even in fully offline environments.

5.1 AI Pipeline Overview

Tier	Provider	Model	Trigger Condition
1 (Primary)	OpenRouter API	google/gemma-4-26b-a4b-it:free	Always attempted first
1 (Fallback)	OpenRouter API	nvidia/nemotron-3-nano-30b-a3b	If primary returns HTTP error
1 (Fallback)	OpenRouter API	google/gemma-4-31b-it:free	If previous model fails
1 (Fallback)	OpenRouter API	nvidia/nemotron-3-ultra-550b	Tertiary OpenRouter fallback
2	Ollama (local)	gemma2:2b @ localhost:11434	If OpenRouter entirely fails
3	Rule Engine	Pure Python, no external calls	If Ollama unavailable / timeout

5.2 OpenRouter API (Primary)

OpenRouter is an API aggregator that provides unified access to dozens of free and paid LLMs. The CDSS uses it as the primary AI provider because it offers large-context models (26B+ parameters) for free, significantly outperforming the local 2B model.

- **Endpoint:** `https://openrouter.ai/api/v1/chat/completions`
- **Auth:** Bearer token in Authorization header (`OPENROUTER_API_KEY` from `.env`)
- **Temperature:** 0.2 — low temperature for deterministic clinical output
- **System prompt:** 'You are a clinical decision support system AI. Respond concisely with exact data.'
- **Thinking models:** payload includes *thinking: {type: disabled}* to suppress chain-of-thought XML that would break JSON parsing
- **Timeout:** 20 seconds per request
- **Retry logic:** Iterates through all models in `OPENROUTER_FALLBACKS` list on failure

5.3 Local Ollama / Gemma2 2B (Secondary Fallback)

If all OpenRouter calls fail, the system falls back to a locally-running Ollama server. This enables completely offline operation in areas with unreliable internet — critical for rural Indian healthcare settings.

- **Endpoint:** `http://localhost:11434/api/generate`
- **Model:** `gemma2:2b` (`OLLAMA_MODEL` env variable, default: `gemma2:2b`)
- **Stream:** False — single synchronous response

- **Timeout:** 60 seconds (longer due to local CPU/GPU inference latency)
- **num_predict:** 1024 tokens max
- **Temperature:** 0.2
- **Error handling:** ConnectionError (Ollama not running), Timeout, generic exceptions all caught and logged

5.4 Rule-Based Engine (Offline Hard Fallback)

A deterministic scoring engine that requires no external services. It matches symptoms and free text against a knowledge base of 11 diseases using keyword matching + weighted scoring.

Disease Knowledge Base (11 conditions):

Disease	Required Keywords	Supportive Keywords	Base Score
Bacterial Meningitis	fever, neck stiff	headache, sensorium, vomit, rash	0.82
Malaria (P. falciparum)	fever, chill	headache, fatigue, vomit, joint	0.79
Dengue Fever	fever	rash, joint, headache, fatigue	0.72
Typhoid Fever	fever	headache, abdominal, diarrh	0.68
Acute Gastroenteritis	diarrh, vomit	fever, abdominal, fatigue	0.74
Acute Coronary Syndrome	chest	breath, fatigue, sweat	0.77
Viral Hepatitis A	jaundice	abdominal, fatigue, fever	0.71
Leptospirosis	fever, joint	jaundice, headache, muscle	0.64
URTI	cough	fever, throat, ear, fatigue	0.67
Acute Viral Fever	fever	fatigue, headache, chill	0.60
Urinary Tract Infection	fever	fatigue, abdominal, dysuria	0.55

Scoring algorithm: `base_score + (supportive_matches × 0.04) + text_direct_match_bonus (0.15) + lab_bonus (0.08–0.10 for abnormal platelets/WBC)`. Scores capped at 0.98.

5.5 NLP Extraction Logic

The `/extract` endpoint converts free-text clinical notes into structured fields. Three implementations are stacked (LLM primary, then regex fallback):

- **LLM Extraction (OpenRouter/Ollama):** Sends structured prompt asking for JSON output with age, sex, symptoms[], vitals{}, duration, chiefComplaint, confidence, rawEntities[].
- **Regex Fallback:** Uses hand-crafted regex patterns for age (`(\d{1,3}s*year)`), sex (`(male|female|M|F)`), duration (`(\d+ day/week/hour/month)`), temperature (`temp:\s+\d{2,3}`), BP (`bp:\s+\d{2,3}/\d{2,3}`), pulse,

- SpO2.
- **Symptom Pattern Dictionary:** 18 symptom categories each with multiple keyword patterns (e.g. 'Fever' matches: fever, febrile, pyrexia, temperature, high temp).
- **Confidence scoring:** $0.5 + (0.08 \times \text{symptom_count}) + 0.1$ if age found + 0.05 if sex found, capped at 0.95 .

5.6 Prompt Engineering

Both prompts are carefully engineered for reliable structured output from small LLMs:

DIAGNOSE PROMPT (excerpt):

"OUTPUT RULE: Reply with ONLY the raw JSON array below. No words before it, no words after it, no markdown fences.

Patient: Free-Text: {free_text} | Symptoms: {symptoms} | Vitals: {vitals}

Required: [{"name": "Disease", "confidence": 0.85, "shap": ["finding1"]}]

Start reply with '[' and end with ']'. Nothing else."

EXTRACT PROMPT (excerpt):

"OUTPUT RULE: Reply with ONLY the raw JSON object below.

Clinical Note: "{text}"

Required: {"age": "45 years", "sex": "male", "symptoms": [...],

"vitals": {"temperature": "", "bp": "", ...},

"chiefComplaint": "...", "confidence": 0.9, "rawEntities": [...]}

Start reply with '{' and end with '}'. Nothing else."

5.7 JSON Extraction Utilities

- **strip_thinking_tags():** Removes <think>...</think> chain-of-thought blocks emitted by reasoning models (Qwen3, DeepSeek-R1) before JSON parsing.
- **extract_json_array():** Robustly finds the first '[' in LLM output, counts bracket depth to find the matching ']', and calls json.loads(). Strips markdown fences first.
- **extract_json_object():** Same algorithm for JSON objects ('{' → '}' matching).

6. Database

6.1 Database Engine & ORM

The database layer uses **SQLAlchemy 2.x ORM** configured in **backend/database.py**. SQLite is the default; PostgreSQL can be activated by setting `DATABASE_URL` in the `.env` file.

Config Item	Value
Default DB	SQLite — file: backend/cdss.db
Optional DB	PostgreSQL — e.g. postgresql://postgres:pass@localhost:5432/cdss
ORM	SQLAlchemy 2.x (declarative_base style)
Session factory	SessionLocal = sessionmaker(autocommit=False, autoflush=False)
SQLite flag	check_same_thread=False — allows FastAPI thread pool to reuse sessions
Connection	DATABASE_URL env var; auto-detects SQLite vs. PostgreSQL by prefix
Table creation	Base.metadata.create_all(bind=engine) runs at app startup

6.2 Schema & Models (backend/models.py)

Column	SQLAlchemy Type	Constraints	Description
id	String	PK, index	Patient ID, format P-XXXXXXXX (UUID)
name	String	index	Patient full name
age	String	—	Age as string (e.g. '45')
sex	String	—	'male' or 'female'
condition	String	—	Primary diagnosis (AI result)
status	String	—	'stable', 'moderate', or 'critical'
time	String	—	Human-readable time string 'Just now'
timestamp	DateTime	default=utcnow	Auto-set to current UTC time on insert

6.3 CRUD Operations

Operation	Endpoint	Logic
Read All	GET /patient/history	Query Consultation ordered by timestamp DESC, map to JSON

Upsert	POST /patient/save	Filter by id; if exists → update fields, else → insert new record
Delete All	DELETE /patient/history	db.query(Consultation).delete(); commit

6.4 Client-Side Fallback Storage (localStorage)

Key: **saved_patients** (JSON array in browser localStorage)

The frontend always writes a copy of each saved patient to localStorage immediately after calling POST /patient/save (even if the backend call fails). On page load, PatientHistory.tsx and Dashboard.tsx fetch from the backend first, then merge any additional records from localStorage that are not already present (de-duplicated by patient id). This dual-write strategy ensures the UI remains responsive and fully functional even when the backend server is temporarily unreachable.

7. Red Flag Detection Engine

The `detect_red_flags()` function in `main.py` is called on every `/predict` request BEFORE the LLM diagnosis to instantly surface life-threatening conditions. It is pure Python — no AI required — ensuring zero latency alerting.

Red Flag Rule	Trigger Condition	Alert Message
Meningitis Risk	fever + neck stiff in symptoms	High Fever + Neck Stiffness → Immediate LP & IV Antibiotics
Cardiac Emergency	chest + breath in symptoms	Chest Pain + Breathlessness → ECG immediately. Call 108
Neuro Emergency	sensorium / altered in symptoms	Altered Sensorium → Rule out meningitis/stroke/hypoglycaemia
Shock (Hypotension)	Systolic BP < 90 mmHg	Hypotension (BP {value}) → IV Fluids. Urgent referral
Hypoxia	SpO2 < 92%	SpO2 {value}% → High-flow oxygen immediately
Hyperpyrexia	Temperature ≥ 40.0°C	Hyperpyrexia ({value}°C) → Risk of febrile seizures
Tachycardia	Pulse > 120 bpm	Tachycardia ({value} bpm) → Rule out sepsis/arrhythmia

■■ **Important:** Red flags are detected server-side and returned in the `redFlags[]` array of the `/predict` response. The frontend renders them as prominent red glass alert cards at the top of the `DiagnosticResults` page. If any red flag is present, the 'Recommended Action' panel automatically shows 'Urgent Hospital Referral' instead of 'Outpatient Treatment'.

8. NHM Treatment Protocol (Knowledge Base / RAG)

After diagnosis, the backend performs a **Retrieval-Augmented Generation (RAG)** step — it looks up the primary diagnosis in the **NHM_TREATMENTS** dictionary and returns evidence-based drug, dosage, duration, and referral guidance. This is a curated static knowledge base, not a vector database, ensuring 100% reliability without an external embedding service.

Disease	Drug 1	Drug 2
Malaria (P. falciparum)	Artemether-Lumefantrine 4 tabs BD × 3 days	Primaquine 0.75 mg/kg single dose
Malaria (P. vivax)	Chloroquine 10 mg/kg × 3 days	Primaquine 0.25 mg/kg × 14 days
Dengue Fever	Paracetamol 500-1000mg q6h	ORS / IV Fluids (maintain hydration)
Typhoid Fever	Azithromycin 500mg OD × 7 days	Ceftriaxone IV 2g OD × 10-14 days
Acute Gastroenteritis	ORS 200-400 mL per stool	Zinc 20mg OD × 14 days (child <5yr)
Bacterial Meningitis	Ceftriaxone IV 2g q12h × 10-14 days	Dexamethasone IV 0.15 mg/kg q6h × 4 days
Acute Coronary Syndrome	Aspirin 325mg stat (chew)	Clopidogrel 300mg loading dose stat
Hypertension	Amlodipine 5-10mg OD	Losartan 50mg OD
Viral Hepatitis A	Supportive care + high-carb diet	Vitamin K 10mg IM × 3 days
Leptospirosis	Doxycycline 100mg BD × 7 days	Penicillin G IV 1.5 MU q6h × 7 days
URTI	Paracetamol 500-1000mg q6-8h PRN	Saline Nasal Drops × 5 days
Acute Viral Fever	Paracetamol 500-1000mg q6-8h	ORS as needed for hydration

If the primary diagnosis does not match any key in **NHM_TREATMENTS**, a **DEFAULT_TREATMENT** response is returned: Paracetamol 500-1000mg q6-8h + ORS with a follow-up-in-48h referral note.

9. End-to-End Workflow

9.1 Consultation Workflow (Main Diagnostic Path)

#	Step	Details
1	Open App	Doctor navigates to http://localhost:5173
2	Navigate	Clicks 'Start AI Diagnostic' → /consultation
3	Enter Note	Types or pastes free-text clinical note in textarea
4	Extract (optional)	Clicks 'Extract with AI' → POST /extract → fields auto-filled
5	Fill Form	Optionally adds/edits symptoms, vitals, lab values, name
6	Submit	Clicks 'Start AI Diagnostic' button
7	POST /predict	Frontend sends full PredictRequest payload to backend
8	NLP on freeText	Backend calls extract_nlp(freeText) to supplement symptoms
9	Red Flag Check	detect_red_flags(symptoms, vitals) → flags[]
10	AI Diagnosis	predict_diagnoses() → OpenRouter → Ollama → Rule Engine
11	Treatment Lookup	NHM_TREATMENTS.get(primaryDiagnosis)
12	Response	Backend returns JSON: patient, diagnoses, redFlags, treatments
13	Navigate	Frontend writes to sessionStorage → navigates to /diagnostic-results
14	Display	DiagnosticResults.tsx renders: alerts, primary dx, chart, treatments

9.2 Data Save Workflow

#	Step	Details
1	View Results	Doctor reviews DiagnosticResults page
2	Click Save	Clicks 'Save Diagnostic to History' button
3	POST /patient/save	Frontend sends {id, name, age, sex, condition, status} to backend
4	DB Upsert	Backend checks if id exists → update or insert in SQLite
5	localStorage	Frontend ALSO writes record to localStorage['saved_patients'] immediately
6	UI Update	Button turns green: 'Saved to History Database'

7	Navigate	Optionally click 'View History Archive →' to go to /history
---	----------	---

9.3 Patient History Workflow

#	Step	Details
1	Navigate	Click 'Patient History' in navbar → /history
2	Fetch Backend	GET /patient/history → SQLAlchemy query → JSON list
3	Merge Local	Frontend merges localStorage['saved_patients'] for records not in DB
4	Display List	Renders patient rows with status badges, diagnoses, last visit
5	Search	Filter by name or patient ID (case-insensitive)
6	Filter Badge	Click 'critical' / 'moderate' / 'stable' to filter by status
7	Click Row	Writes summary to sessionStorage, navigates to /results
8	View Results	Results.tsx shows the historical diagnostic record (read-only)

10. Environment & Configuration

File	Variable	Default / Example	Purpose
backend/.env	OPENROUTER_API_KEY	sk-or-v1-...	Auth token for OpenRouter REST API
backend/.env	OPENROUTER_MODEL	google/gemma-4-26b-a4b-it:free	Primary LLM model to use
backend/.env	DATABASE_URL	sqlite:///./cdss.db	DB connection string (omit for SQLite default)
backend/.env	OLLAMA_URL	http://localhost:11434	Ollama server base URL
backend/.env	OLLAMA_MODEL	gemma2:2b	Local Ollama model name
frontend/.env	VITE_BACKEND_URL	http://localhost:8000	Backend URL used by React app
frontend/.env	VITE_OPENROUTER_API_KEY	sk-or-v1-...	Stored in frontend (not currently used in prod)

■ Files Structure:

```
CDSS/
├── backend/
│   ├── main.py          # FastAPI app + AI logic + routes (911 lines)
│   ├── database.py      # SQLAlchemy engine + session factory
│   ├── models.py        # Consultation ORM model
│   ├── requirements.txt  # Python dependencies
│   ├── cdss.db          # SQLite database file (auto-created)
│   ├── .env             # OpenRouter API key + model config
│   └── test_gemma.py    # Dev script to test Ollama connection
├── frontend/
│   ├── src/
│   │   ├── App.tsx      # React Router config (6 routes)
│   │   ├── index.css    # Full design system (344 lines)
│   │   ├── main.tsx     # React 19 createRoot entry
│   │   └── components/
│   │       ├── Layout.tsx # Shared navigation + background shell
│   │       └── pages/    # (6 pages)
│   ├── .env             # VITE_BACKEND_URL
│   └── package.json     # React 19 + Vite 8 + Recharts + Lucide
```

11. Security Considerations

Area	Current State	Recommended Improvement
Authentication	None — all endpoints are public	Add JWT Bearer token auth (FastAPI-Users or custom)
CORS	allow_origins=["*"] (wide open)	Restrict to specific frontend domain in production
API Keys	Stored in .env files (not committed)	Use a secrets manager (e.g. AWS Secrets Manager) in production
Input Validation	Pydantic models validate field types	Add length limits, field sanitization, SQL injection check
HTTPS	HTTP only in development	Configure TLS/HTTPS via reverse proxy (nginx/Caddy) in production
Rate Limiting	Not implemented	Add slowapi middleware to prevent API abuse
Patient Data	No encryption at rest (SQLite)	Use PostgreSQL with encrypted volumes in production
Session Storage	Diagnostic results in sessionStorage	sessionStorage is cleared on tab close; acceptable for ephemeral data

12. Known Limitations & Roadmap

Current Limitations

- No authentication — any user with network access can call all endpoints.
- CORS is fully open (allow_origins=[*]) — not safe for production deployment.
- The Dashboard's weekly chart uses **static mock data**, not real time-series queries.
- Patient visit count is always hardcoded to 1 — no true visit tracking.
- No ML model training — the rule-based fallback is keyword-matching, not a trained classifier.
- The OPENROUTER_API_KEY in frontend/.env is currently unused — all AI calls go through backend.
- No audit trail — there is no logging of who accessed which patient record.
- SQLite is single-writer — concurrent writes from multiple staff may cause locking issues.

Roadmap & Future Enhancements

- **Phase 1:** Add JWT authentication, role-based access (admin / doctor / nurse).
- **Phase 1:** Switch CORS to production domain only; add rate limiting.
- **Phase 2:** Replace SQLite with PostgreSQL; add Alembic migrations.
- **Phase 2:** Implement real visit-count tracking; multi-visit patient timeline.
- **Phase 3:** Train an XGBoost or LightGBM classifier on labelled symptom-disease data to replace the rule-based fallback.
- **Phase 3:** Integrate a proper vector database (ChromaDB / Pinecone) for semantic RAG over a larger NHM guideline corpus.
- **Phase 4:** Add voice input for free-text notes (Web Speech API).
- **Phase 4:** Generate a printable PDF diagnostic report per patient consultation.
- **Phase 5:** Integrate with India's ABDM / ABHA health ID system for national patient record linkage.
- **Phase 5:** Multi-language support (Hindi, Tamil, Marathi) for rural clinics.